

Informe Prueba 3 fullstack-Proyecto Optibeans

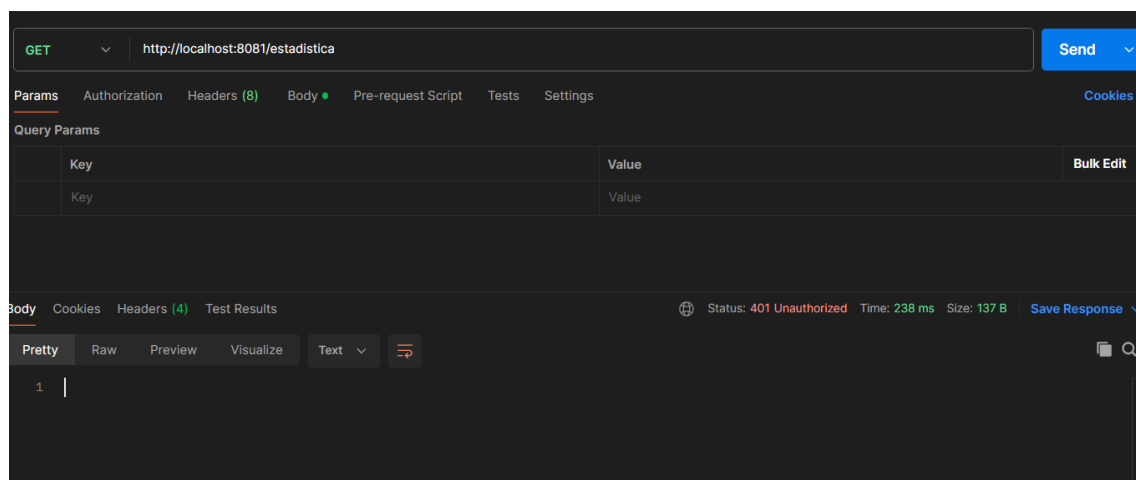
Servicio de autenticación:

El servicio de autenticación es un microservicio aparte que consiste en encargarse de validar a un usuario antes de darle acceso a espacios protegidos por nuestro sistema

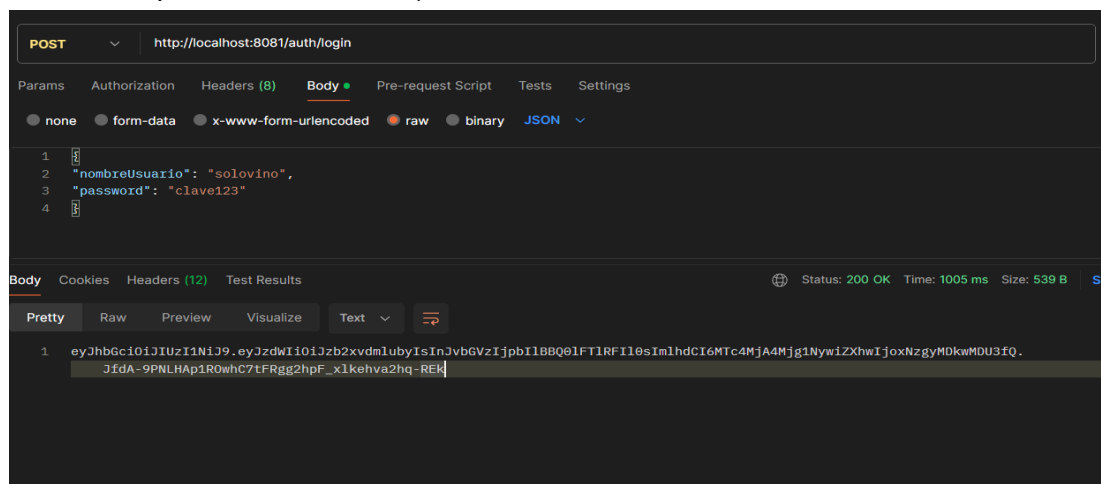
¿Cómo se implementa?

Implementamos JWT (JSON Web Token) con el siguiente flujo

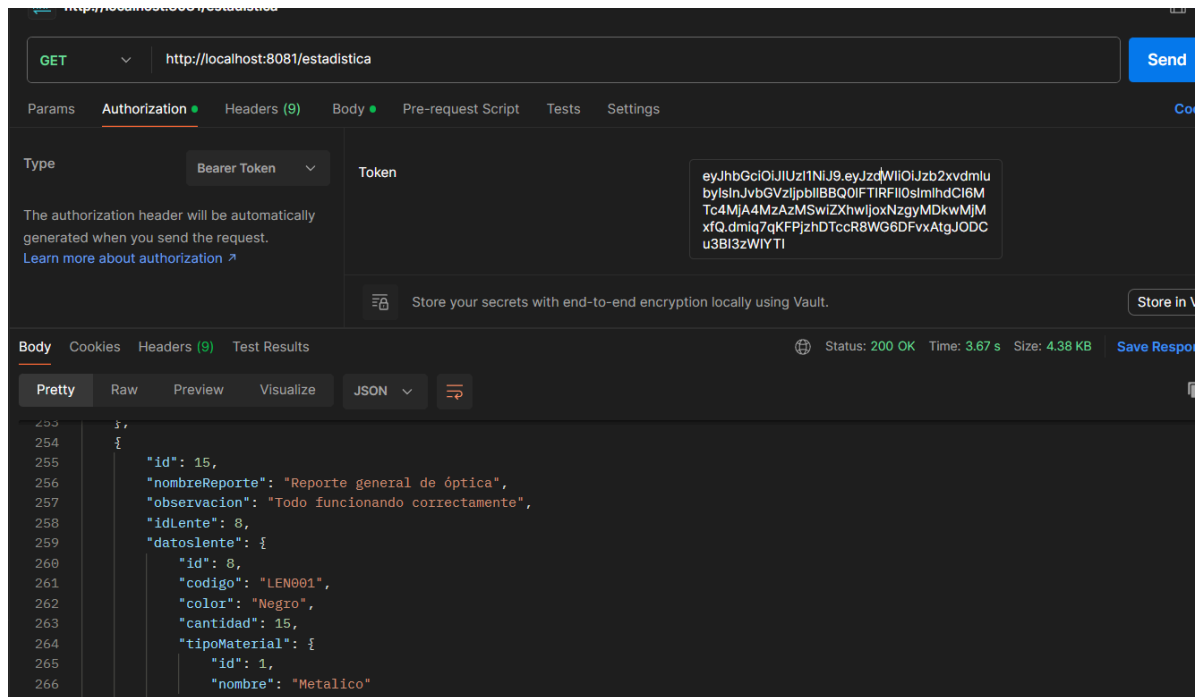
Al intentar ingresar al microservicio “Estadística” sin nada nos saldrá el siguiente mensaje:
401 Unauthorized



Para que funcione debemos ingresar con nuestro usuario y contraseña (Registrar uno si no lo tenemos previamente creado)



Aquí se nos entrega nuestro token y viene el momento de implementarlo



Implementamos nuestro token y ahora s\u00ed nos deja acceder al microservicio Estad\u00edstica

Implementaci\u00f3n por capas:

Para explicar esto usaremos el microservicio “Ventas” el cual construimos mediante una arquitectura por capas (model-repository-service-controller)

Controller:

Es el traductor recibe las solicitudes http y responde la petici\u00f3n, ejemplo en c\u00f3digo:

```
@GetMapping("/{id}")
public Venta obtener (@PathVariable Long id) {
    return ventaService.obtenerVentaCompleta(id);
}
```

Service:

La capa service se encarga de guardar la l\u00f3gica de negocio, ejemplo en c\u00f3digo:

```
public Venta guardarVenta(Venta venta) {
    Venta guardada = ventaRepository.save(venta);
    return enriquecerConClienteYLente(guardada);
}
```

Repository:

Es el que se comunica con la base de datos, ejemplo en c\u00f3digo

```
public interface VentaRepository extends JpaRepository<Venta, Long> {
}
```

Model:

Aquí es donde se representa la entidad/tabla creada en la base de datos, ejemplo en código:

```
public class Venta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String fechaVenta;
    private int monto;
    private String metodoPago;

    private Long idCliente;
    private Long idLente;

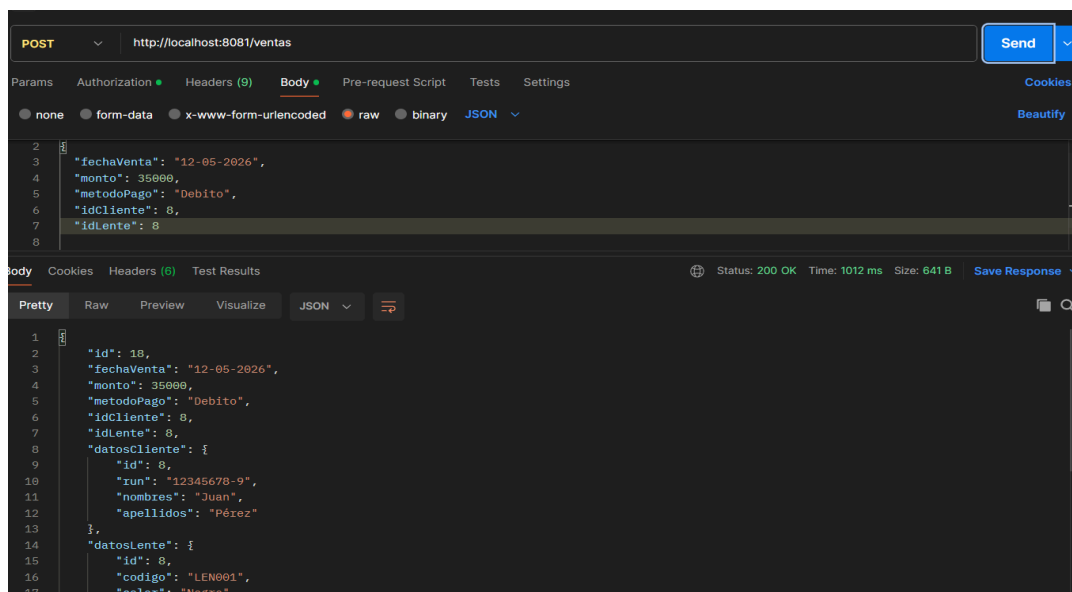
    @Transient
    private Object datosCliente;

    @Transient
    private Object datosLente;

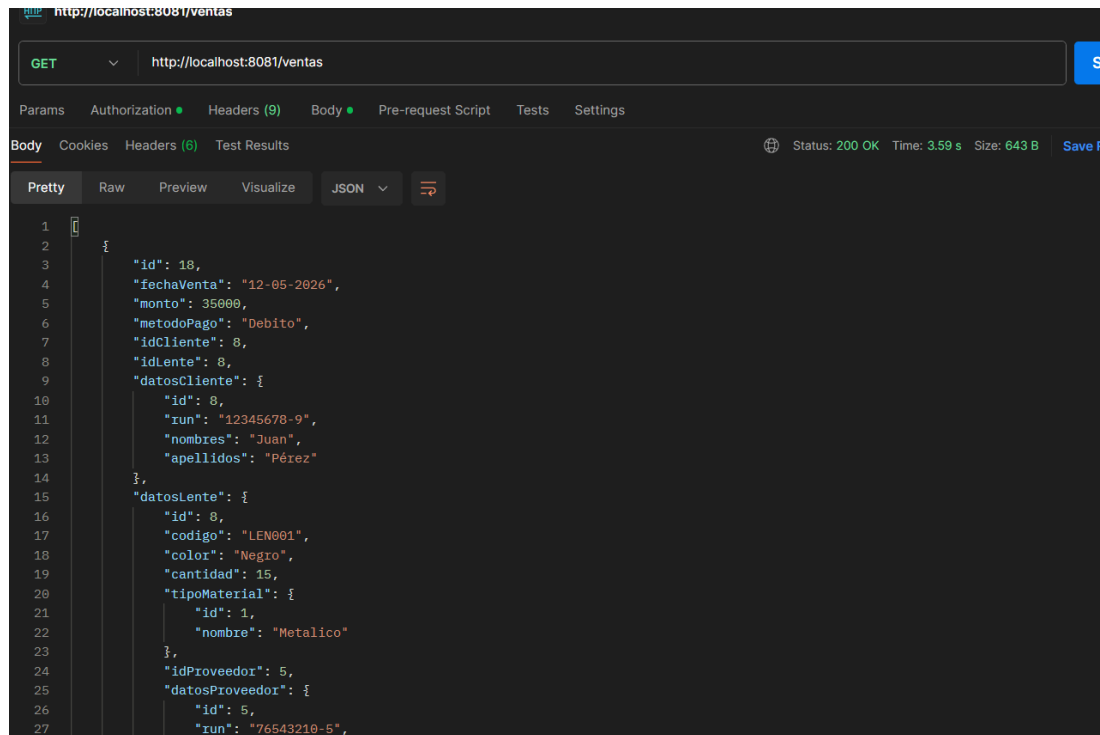
}
```

Evidencias de pruebas en Postman del funcionamiento:

Post: creación de la venta:

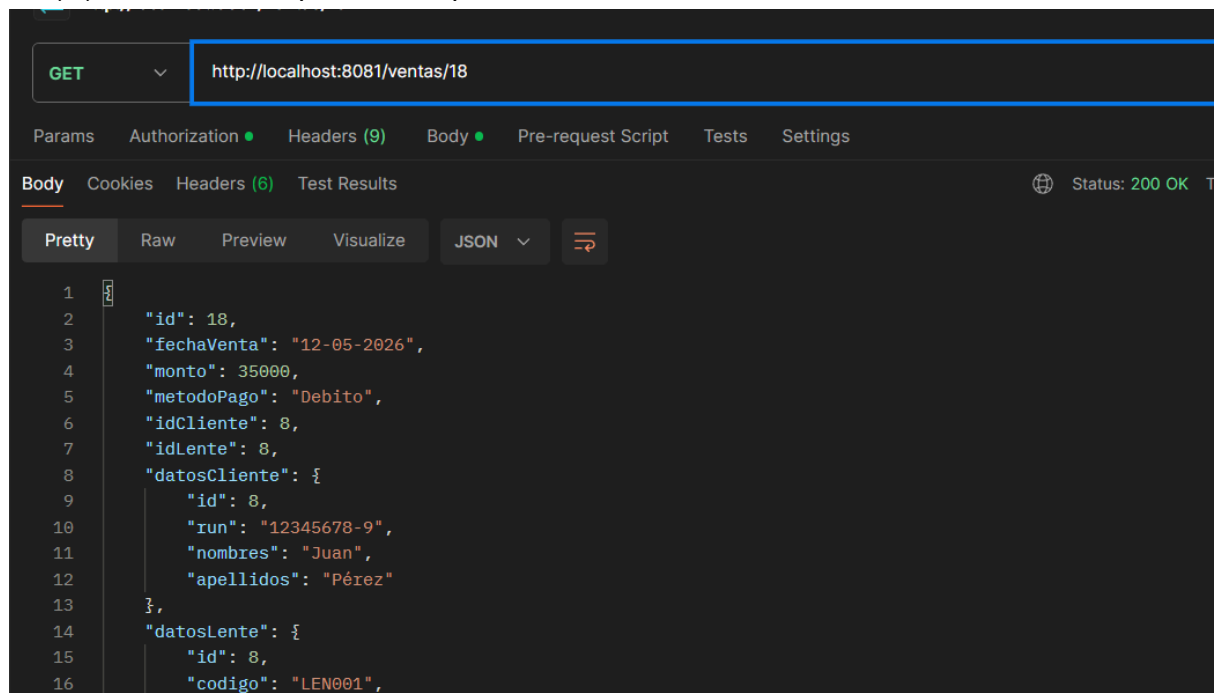


Get- trae todas las ventas creadas:



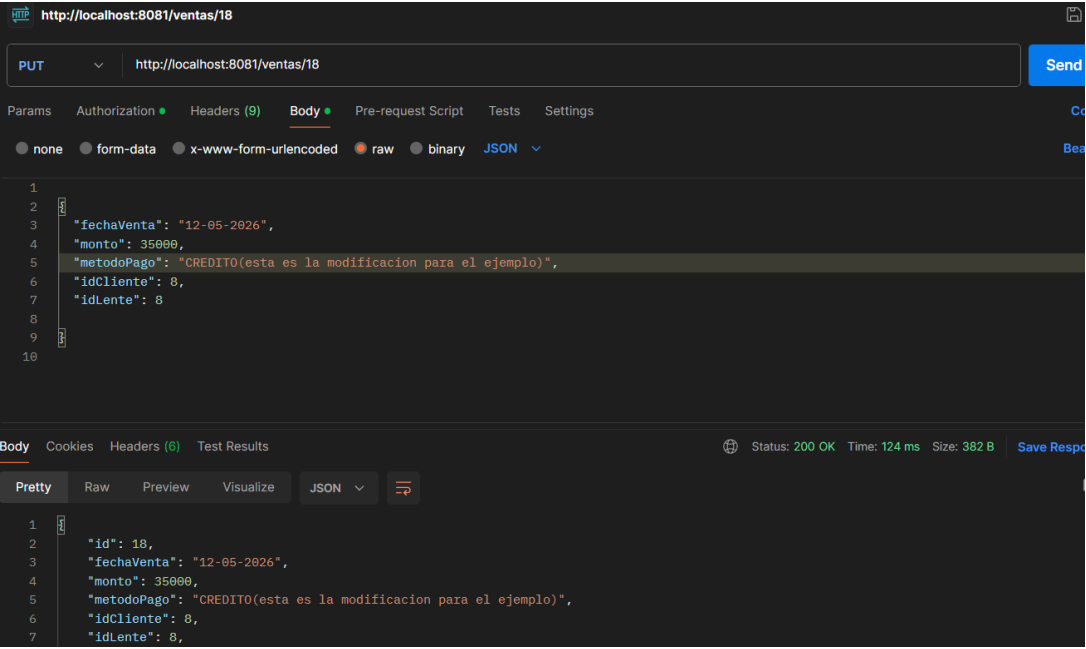
```
1 [{"id": 18,  
2   "fechaVenta": "12-05-2026",  
3   "monto": 35000,  
4   "metodoPago": "Debito",  
5   "idCliente": 8,  
6   "idLente": 8,  
7   "datosCliente": {  
8     "id": 8,  
9     "run": "12345678-9",  
10    "nombres": "Juan",  
11    "apellidos": "Pérez"  
12  },  
13  "datosLente": {  
14    "id": 8,  
15    "codigo": "LEN001",  
16    "color": "Negro",  
17    "cantidad": 15,  
18    "tipoMaterial": {  
19      "id": 1,  
20      "nombre": "Metalico"  
21    },  
22    "idProveedor": 5,  
23    "datosProveedor": {  
24      "id": 5,  
25      "run": "76543210-5",  
26    }  
27  }  
28  }  
29  ]
```

Get(id) trae una venta por un id específico:

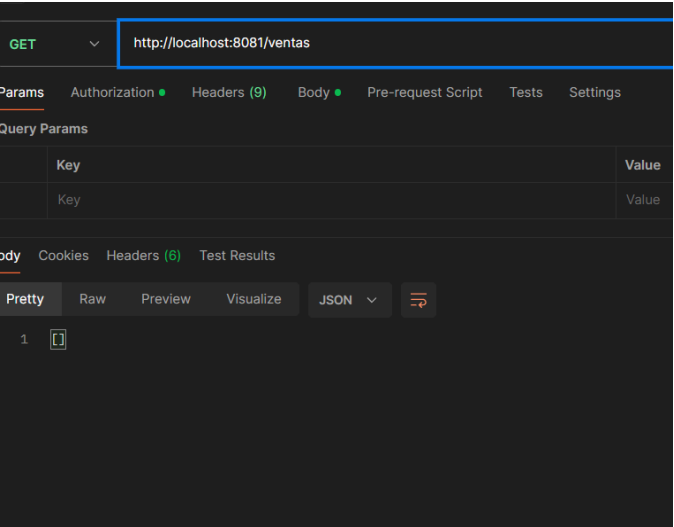
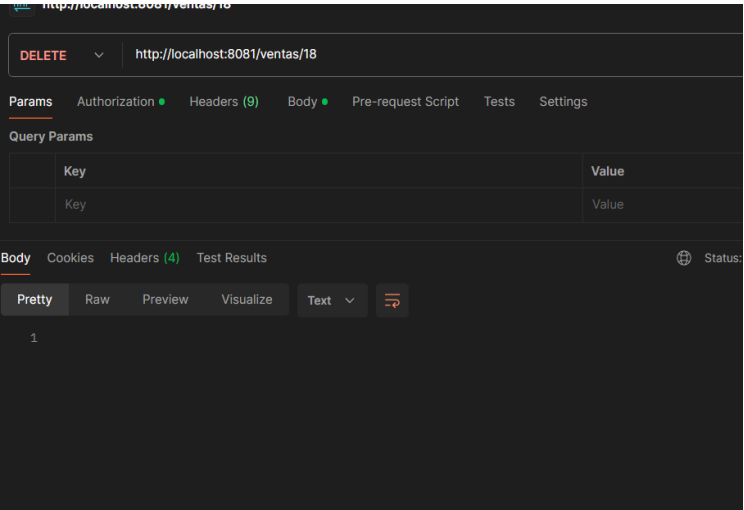


```
1 {"id": 18,  
2   "fechaVenta": "12-05-2026",  
3   "monto": 35000,  
4   "metodoPago": "Debito",  
5   "idCliente": 8,  
6   "idLente": 8,  
7   "datosCliente": {  
8     "id": 8,  
9     "run": "12345678-9",  
10    "nombres": "Juan",  
11    "apellidos": "Pérez"  
12  },  
13  "datosLente": {  
14    "id": 8,  
15    "codigo": "LEN001",  
16    "color": "Negro",  
17    "cantidad": 15,  
18    "tipoMaterial": {  
19      "id": 1,  
20      "nombre": "Metalico"  
21    },  
22    "idProveedor": 5,  
23    "datosProveedor": {  
24      "id": 5,  
25      "run": "76543210-5"  
26    }  
27  }  
28  }  
29  }
```

Put(id) modifica/actualiza una venta:



Delete elimina la venta:



Como se pudo notar al hacer el get en ventas este traía información tanto de lentes como de cliente esto se debe a la comunicación entre microservicios esto lo conseguimos gracias a la utilización de WebClient, cuando se consulta una venta esta hace llamadas http a los otros microservicios con este WebClient, evidencia en código:

```
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient.Builder webClientBuilder() {
        return WebClient.builder();
    }
}

private Venta enriquecerConClienteYLente(Venta venta) {

    if(venta.getIdCliente() != null){

        try{

            Object cliente =
                webClientBuilder.build()
                    .get()
                    .uri("http://localhost:8082/clientes/" +
venta.getIdCliente())
                    .retrieve()
                    .bodyToMono(Object.class)
                    .block();

            venta.setDatosCliente(cliente);

        }catch(Exception e){

            venta.setDatosCliente(
                "Informacion del cliente no disponible"
            );
        }
    }

    if(venta.getIdLente() != null){

        try{
```

```

        Object lente =
            webClientBuilder.build()
                .get()
                .uri("http://localhost:8084/api/v1/lentes/"
                    + venta.getIdLente())
                .retrieve()
                .bodyToMono(Object.class)
                .block();

        venta.setDatosLente(lente);

    } catch (Exception e) {

        venta.setDatosLente(
            "Informacion del lente no disponible"
        );
    }
}

return venta;
}

```

Swagger:

¿Qué es swagger y en qué consiste?:

Swagger es un conjunto de herramientas de código abierto que ayudan a los desarrolladores a diseñar, construir, documentar y consumir servicios web RESTful (APIs)

¿Cómo se implementa?:

En un proyecto de microservicios spring boot swagger se implementa mediante la librería Springdoc OpenAPI

Documentación de código:

```

6
7 spring:
8   application:
9     name: api-gateway
10
11   cloud:
12     gateway:
13
14       # --- CONFIGURACIÓN DE CORS GLOBAL ---
15     globalcors:
16       cors-configurations:
17         '[/*]':
18           allowedOrigins: "*"
19           allowedMethods:
20             - GET
21             - POST
22             - PUT
23             - DELETE
24             - OPTIONS
25           allowedHeaders: "*"
26
27   routes:

```

```

- id: auth-service
# Rutas doocccc

- id: clientes-docs
  uri: http://localhost:8082
  predicates:
    - Path=/clientes-docs

- id: proveedor-docs
  uri: http://localhost:8083
  predicates:
    - Path=/proveedor-docs

- id: lentes-docs
  uri: http://localhost:8084
  predicates:
    - Path=/lentes-docs

- id: consulta-docs
  uri: http://localhost:8085
  predicates:
    - Path=/consulta-docs

- id: receta-docs
  uri: http://localhost:8086
  predicates:
    - Path=/receta-docs

```

```

0
1 eureka:
2   client:
3     register-with-eureka: false
4     fetch-registry: false
5
6 springdoc:
7   swagger-ui:
8     urls:
9     - name: Servicio Envio
10       url: /envio-docs
11     - name: Servicio Consulta
12       url: /consulta-docs
13     - name: Servicio Clientes
14       url: /clientes-docs
15     - name: Servicio Proveedor
16       url: /proveedor-docs
17     - name: Servicio Inventario
18       url: /lentes-docs
19     - name: Servicio Recetas
20       url: /receta-docs
21     - name: Servicio Ventas
22       url: /ventas-docs
23     - name: Servicio Medicos
24       url: /medicos-docs

```

```

@RestController
@RequestMapping("/ventas")
@CrossOrigin(origins = "*")
@Tag(name = "Ventas", description = "Operaciones relacionadas con las
ventas de los lentes")
public class VentaController {

    @Autowired
    private VentaService ventaService;

    @PostMapping
    @Operation(summary = "crea una venta")
    public Venta crear(@RequestBody Venta venta) {
        return ventaService.guardarVenta(venta);
    }
}

public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("API Ventas | Servicio de ventas")
                .version("1.0")

```

```

        .description("Documentación centralizada del Sistema de
ventas"))

        .servers(List.of(
            new
Server().url("http://localhost:8081").description("Servidor a través
del Gateway")
        ));
    }

```

```

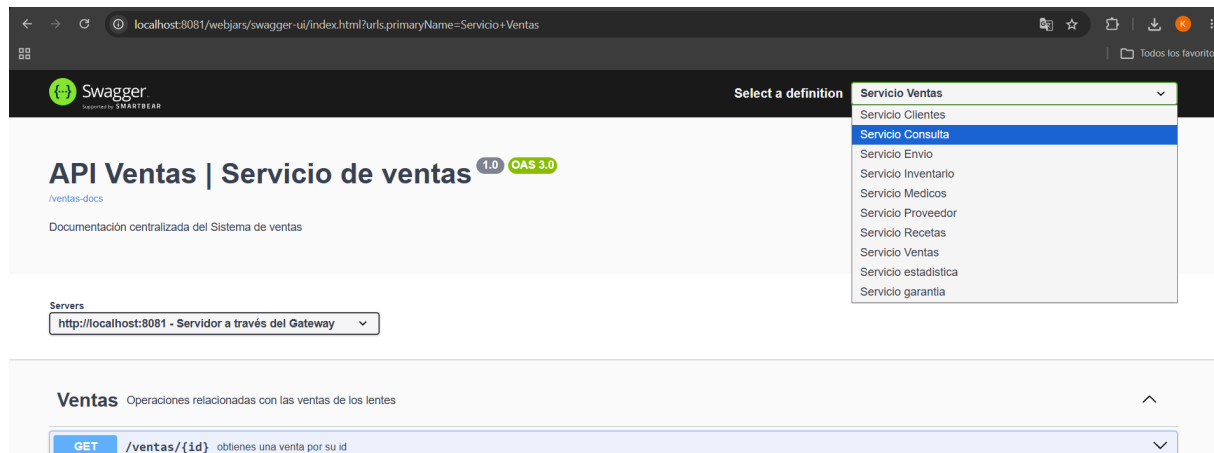
@Configuration
public class WebConfig implements WebMvcConfigurer{

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/*")
            .allowedOrigins("*") // En producción, aquí iría la URL
del Gateway
            .allowedMethods("GET", "POST", "PUT", "DELETE",
"OPTIONS")
            .allowedHeaders("*");
    }
}

```

Ejemplo de ejecución:

Se logra entrar a la página desde la ruta del api gateway logrando mostrar todos los microservicios:



Se logra ejecutar correctamente los microservicios requeridos:

API Ventas | Servicio de ventas 1.0 OAS 3.0

/ventas-docs

Documentación centralizada del Sistema de ventas

Servers

http://localhost:8081 - Servidor a través del Gateway

Ventas Operaciones relacionadas con las ventas de los lentes

GET	/ventas/{id}	obtienes una venta por su id
PUT	/ventas/{id}	Actualizar estadística
DELETE	/ventas/{id}	Elimina una venta
GET	/ventas	lista toda las ventas
POST	/ventas	crea una venta

POST

/ventas

crea una venta

Parameters

No parameters

Request body required

application/json

```
{
  "fechaVenta": "12-03-2026",
  "monto": 20000,
  "metodoPago": "Efectivo",
  "idCliente": 8,
  "idLente": 8
}
```

Execute

Clear

Code

Details

200

Response body

```
{
  "id": 19,
  "fechaVenta": "12-03-2026",
  "monto": 20000,
  "metodoPago": "Efectivo",
  "idCliente": 8,
  "idLente": 8,
  "datosCliente": {
    "id": 8,
    "run": "12345678-9",
    "nombres": "Juan",
    "apellidos": "Pérez"
  },
  "datosLente": {
    "id": 8,
    "codigo": "LEN001",
    "color": "Negro",
    "cantidad": 15,
    "tipoMaterial": {
      "id": 1,
      "nombre": "Metalico"
    },
    "idProveedor": 5,
    "datosProveedor": {
      "id": 5,
      "run": "76543210-5",
      "nombre": "OptiVision Chile",
      "telefono": "909090909"
    }
  }
}
```

Response headers

Download

API Garantias | Servicio de Garantias

1.0 OAS 3.0

[/garantia-docs](#)

Documentación centralizada del Sistema de garantías

Servers

http://localhost:8081 - Servidor a través del Gateway

Garantia

Garantía del lente vendido

GET

/garantias/{id}

obtener una garantía por su id

PUT

/garantias/{id}

actualizar una garantía

DELETE

/garantias/{id}

eliminar una garantía

GET

/garantias

lista de todas las garantías

Garantia

Garantía del lente vendido

GET

/garantias/{id}

obtener una garantía por su id

Parameters

Cancel

Name	Description
id * required	
integer(\$int64)	4
(path)	

Execute Clear

http://localhost:8081/garantias/4

Server response

Code

Details

200

Response body

```
{
  "id": 4,
  "motivo": "Daño en el armazon",
  "estado": "En proceso",
  "fechasolicitud": "10/06",
  "resolucion": "A confirmar",
  "idventa": 15,
  "datosVenta": {
    "id": 15,
    "fechaVenta": "12-03-2026",
    "monto": 20000,
    "metodoPago": "Efectivo",
    "idCliente": 5,
    "idlente": 8,
    "datosCliente": {
      "id": 5,
      "run": "12345678-9",
      "nombres": "Juan",
      "apellidos": "Pérez"
    }
  },
  "datosLente": {
    "id": 8,
    "codigo": "LEN001",
    "color": "Negro",
    "cantidad": 15,
    "tipoMaterial": {
      "id": 1,

```

Download

Swagger
Supporting SMARTBEAR

Select a definition **Servicio Envio**

API Envio | Servicio de envio 1.0 OAS 3.0

/envio-docs

Documentación centralizada del Sistema de envio

Servers
http://localhost:8081 - Servidor a través del Gateway

Envio Operaciones relacionadas con los Envios de la optica

- GET** /envio/{id} obtener un envio por su id
- PUT** /envio/{id} Actualizar un envio
- GET** /envio lista de todos los envios

Request URL
http://localhost:8081/envio

Server response

Code Details

200

Response body

```
{
  "idEnvio": 1,
  "fechaDespacho": "06-07-2026",
  "fechaEntrega": "08-07-2026",
  "direccion": "Av. Esq. Blanca 501, 9251863 Maipú, Región Metropolitana",
  "estado": "Saliendo",
  "notificacion": {
    "idNotificacion": 1,
    "mensaje": "Su paquete esta en camino",
    "envioMensaje": "Enviado",
    "estadoNotificacion": "Leído"
  },
  "idVenta": 19,
  "datosVenta": {
    "id": 19,
    "fechaVenta": "12-03-2026",
    "monto": 20000,
    "metodoPago": "Efectivo",
    "idCliente": 8,
    "datosCliente": {
      "id": 8,
      "run": "12345678-9",
      "nombres": "Juan",
      "apellidos": "Pérez"
    }
  }
}
```

Response headers

```
content-type: application/json
date: Mon, 22 Jun 2026 03:40:13 GMT
```

Testing:

¿En qué consiste?:

El testing es un proceso técnico que sirve para evaluar la calidad y comportamiento de un sistema. Su relevancia consiste en que gracias a estas pruebas podemos encontrar fallos a solucionar y verificar que el programa cumpla con los requerimientos de negocio especificados

Pruebas unitarias:

Son el primer nivel de las pruebas, centradas en la unidad más pequeña de código de forma aislada (funciones, métodos, clases), con esto nos aseguramos de que cada pieza funcione

correctamente de forma aislada sin depender de que la base de datos o el otro microservicio estén encendidos.

Ejemplos con evidencia:

Proveedor: se testea que la capa proveedorService sea capaz de ejecutar su lógica de guardado independiente, sin una conexión red o base de datos

Se verifica que el método no altere datos críticos del negocio como el run durante el proceso de transferencia al repositorio

Se testea el comportamiento y eficiencia de flujo:

```
package com.optica.service_proveedor;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import com.optica.service_proveedor.model.Proveedor;
import com.optica.service_proveedor.repository.Proveedorrepository;
import com.optica.service_proveedor.service.ProveedorService;

@ExtendWith(MockitoExtension.class)
public class ProveedorServiceTest {

    @Mock
    private Proveedorrepository proveedorrepository;

    @InjectMocks
    private ProveedorService proveedorService;

    @Test
    @DisplayName("Deberia guardar a un proveedor correctamente")
    void guardarproveedortest(){
        Proveedor proveedor = new Proveedor();
        proveedor.setNombre("lyu limitada");
        proveedor.setRun("76607348-4");
```

```

        when(proveedorrepository.save(any(Proveedor.class)))
            .thenReturn(invocation -> {
                Proveedor p = invocation.getArgument(0);
                p.setId(1L);
                return p;
            });

        Proveedor resultado = proveedorService.guardar(proveedor);
        assertNotNull(resultado);
        assertEquals(1L, resultado.getId());
        assertEquals("lyu limitada", resultado.getNombre());

        verify(proveedorrepository, times(1)).save(proveedor);
    }
}

```

The screenshot shows an IDE with a Java file named `ProveedorServiceTest`. The code includes imports for `Proveedor`, `Proveedorrepository`, and `ProveedorService`. It uses `@ExtendWith(MockitoExtension.class)` and `@Mock` to create a mock repository. The `guardarproveedortest()` method creates a `Proveedor` object with a name of "lyu limitada" and a run number of "76607348-4", then calls `guardar()` on the `ProveedorService`. The IDE's `TEST RESULTS` tab shows the test passed successfully. The `Test Runner for Java` panel on the right shows the test execution details, including the class `ProveedorServiceTest` and the method `guardarproveedortest()`.

```

17 import com.optica.service_proveedor.model.Proveedor;
18 import com.optica.service_proveedor.repository.Proveedorrepository;
19 import com.optica.service_proveedor.service.ProveedorService;
20
21 @ExtendWith(MockitoExtension.class)
22 public class ProveedorServiceTest {
23     @Mock
24     private Proveedorrepository proveedorrepository;
25     @InjectMocks
26     private ProveedorService proveedorService;
27     @Test
28     @DisplayName("Deberia guardar a un proveedor correctamente")
29     void guardarproveedortest(){
30         Proveedor proveedor = new Proveedor();
31         proveedor.setNombre(nombre: "lyu limitada");
32         proveedor.setRun(run: "76607348-4");
33         proveedorrepository.save(any(Proveedor.class));
34     }
}

```

PROBLEMS 144 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

%TESTC 1 v2
 %TSTTREE2,com.optica.service_proveedor.ProveedorServiceTest,true,1,false,1,ProveedorServiceTest,,[engine:junit-jupiter]/[class:com.optica.service_proveedor.ProveedorServiceTest]
 %TSTTREE3,guardarproveedortest(com.optica.service_proveedor.ProveedorServiceTest),false,1,false,2,Deberia guardar a un proveedor correctamente,,[engine:junit-jupiter]/[class:com.optica.service_proveedor.ProveedorServiceTest]/[method:guardarproveedortest()]
 %TESTS 3,guardarproveedortest(com.optica.service_proveedor.ProveedorServiceTest)

Test Runner for Java

- ✓ ProveedorServiceTest \$(symbol-na..
- ✓ guardarproveedortest() \$(symbol-..

> 5 older results

Cliente:

Se está testeando transición de estado del objeto: se evalúa que que el servicio reciba el objeto transitorio y devuelva correctamente un objeto persistido

Se testea que el método guardar de cliente service funcione sin una conexión real

Se está probando el comportamiento de llaves primarias auto-incrementables

```

package com.optica.service_cliente;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import com.optica.service_cliente.model.Cliente;
import com.optica.service_cliente.repository.ClienteRepository;
import com.optica.service_cliente.service.ClienteService;

@ExtendWith(MockitoExtension.class)
public class ClienteServiceTest {

    @Mock
    private ClienteRepository clienteRepository;

    @InjectMocks
    private ClienteService clienteService;

    @Test
    @DisplayName("Deberia guardar a un cliente correctamente")
    void guardarclientetest() {
        Cliente cliente = new Cliente();
        cliente.setNombres("Charlie bradbury");
        cliente.setRun("14966343-4");

        when(clienteRepository.save(any(Cliente.class)))
            .thenAnswer(invocation -> {
                Cliente p = invocation.getArgument(0);
                p.setId(1L);
                return p;
            });

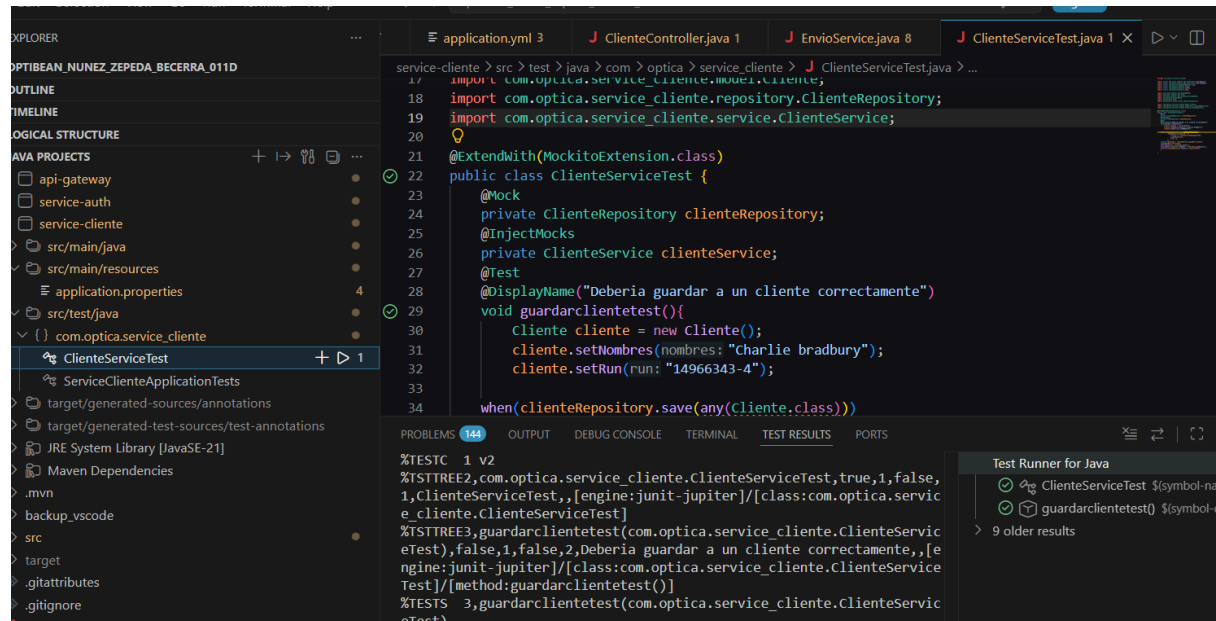
        Cliente resultado = clienteService.guardar(cliente);
        assertNotNull(resultado);
        assertEquals(1L, resultado.getId());
        assertEquals("Charlie bradbury", resultado.getNombres());
    }
}

```

```

        verify(clienteRepository,times(1)).save(cliente);
    }
}

```



Receta:

Se testea la capacidad del servicio de fusionar dos fuentes de información independientes

Se comprueba el comportamiento de servicio ante llamadas http

```

package com.optica.service_receta;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import java.util.Optional; // 1. Agrega este import si no se
auto-importa

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

```

```

import org.springframework.web.reactive.function.client.WebClient;
import
org.springframework.web.reactive.function.client.WebClient.ResponseSpec
;

import com.optica.service_receta.model.ConsultaDTO;
import com.optica.service_receta.model.Receta;
import com.optica.service_receta.repository.RecetaRepository;
import com.optica.service_receta.service.RecetaService;

import reactor.core.publisher.Mono;

@ExtendWith(MockitoExtension.class)
public class RecetaServiceTest {

    @Mock
    private RecetaRepository recetaRepository;

    @Mock
    private WebClient.Builder webclientBuilder;

    @InjectMocks
    private RecetaService recetaService;

    @Test
    void obtenerRecetaCompletaTest() {
        // === ARRANGE ===

        Long recetaId = 1L;
        Long ConsultaId = 100L;

        Receta recetaMock = new Receta();
        recetaMock.setId(recetaId);
        recetaMock.setIdConsulta(ConsultaId);

        ConsultaDTO consultaMock = new ConsultaDTO();
        consultaMock.setId(ConsultaId);
        consultaMock.setDiagnostico("Miopia");

        WebClient webClient = Mockito.mock(WebClient.class);
        WebClient.RequestHeadersUriSpec uriSpec =
Mockito.mock(WebClient.RequestHeadersUriSpec.class);
        WebClient.RequestHeadersSpec headersSpec =
Mockito.mock(WebClient.RequestHeadersSpec.class);
        ResponseSpec responseSpec = Mockito.mock(ResponseSpec.class);
        // Ajustado según tu captura de WebClient

```

```
        when(webclientBuilder.build()).thenReturn(webClient);
        when(webClient.get()).thenReturn(uriSpec);
        when(uriSpec.uri(anyString())).thenReturn(headersSpec);
        when(headersSpec.retrieve()).thenReturn(responseSpec);

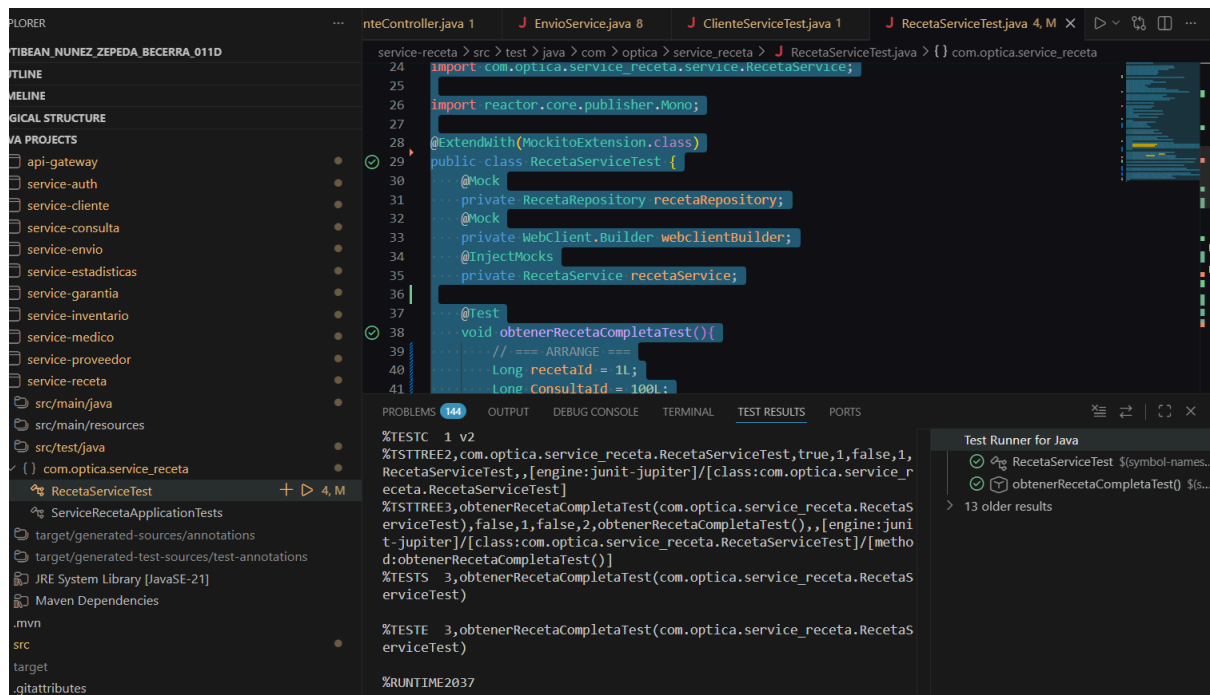
when(responseSpec.bodyToMono(Object.class)).thenReturn(Mono.just(consultaMock));

when(recetaRepository.findById(recetaId)).thenReturn(Optional.of(recetaMock));

        Receta resultado =
recetaService.obtenerRecetaCompleto(recetaId);

        assertNotNull(resultado, "La receta retornada no debería ser nula");
        assertNotNull(resultado.getDatosConsulta(), "los datos de consulta deben estar presentes");

        ConsultaDTO datosRetornados = (ConsultaDTO)
resultado.getDatosConsulta();
        assertEquals("Miopia", datosRetornados.getDiagnostico(), "El diagnostico debe coincidir con el simulado");
        assertEquals(ConsultaId, datosRetornados.getId());
        verify(recetaRepository, times(1)).findById(recetaId);
    }
}
```



Médico:

El test garantiza que el servicio sea capaz de interactuar con la base de datos local:

```
package com.optica.service_medico;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import java.util.Optional;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.reactive.function.client.WebClient;

import com.optica.service_medico.model.ConsultaDTO;
```

```
import com.optica.service_medico.model.Medico;
import com.optica.service_medico.repository.MedicoRepository;
import com.optica.service_medico.service.MedicoService;

import reactor.core.publisher.Mono;

@ExtendWith(MockitoExtension.class)
public class MedicoServiceTest {

    @Mock
    private MedicoRepository medicoRepository;

    @Mock
    private WebClient.Builder webClientBuilder;

    @InjectMocks
    private MedicoService medicoService;

    @Test
    @DisplayName("Debería obtener un médico con su consulta externa de forma completa")
    void obtenerMedicoCompletoTest() {

        Long medicoId = 1L;
        Long consultaId = 100L;

        Medico medicoMock = new Medico();
        medicoMock.setId(medicoId);
        medicoMock.setIdConsulta(consultaId);

        ConsultaDTO consultaMock = new ConsultaDTO();
        consultaMock.setId(consultaId);
        consultaMock.setDiagnostico("astigmatismo");

        WebClient webClient = Mockito.mock(WebClient.class);
        WebClient.RequestHeadersUriSpec uriSpec =
Mockito.mock(WebClient.RequestHeadersUriSpec.class);
        WebClient.RequestHeadersSpec headersSpec =
Mockito.mock(WebClient.RequestHeadersSpec.class);
        WebClient.ResponseSpec responseSpec =
Mockito.mock(WebClient.ResponseSpec.class);
```

```
        when(webClientBuilder.build()).thenReturn(webClient);
        when(webClient.get()).thenReturn(uriSpec);
        when(uriSpec.uri(anyString())).thenReturn(headersSpec);
        when(headersSpec.retrieve()).thenReturn(responseSpec);

when(responseSpec.bodyToMono(Object.class)).thenReturn(Mono.just(consultaMock));

when(medicoRepository.findById(medicoId)).thenReturn(Optional.of(medicoMock));

        Medico resultado =
medicoService.obtenerMedicoCompleto(medicoId);

        assertNotNull(resultado, "El médico retornado no debería ser nulo");
        assertNotNull(resultado.getDatosConsulta(), "los datos de la consulta deben estar presentes");

        ConsultaDTO datosretornados = (ConsultaDTO)
resultado.getDatosConsulta();
        assertEquals("astigmatismo", datosretornados.getDiagnostico(),
"el diagnostico debe coincidir con el simulado");
        assertEquals(consultaId, datosretornados.getId());

        verify(medicoRepository, times(1)).findById(medicoId);
    }
}
```

